

Contabo Server — New Project Deployment Guide

Host pattern used by EasyFuel / Inspect360 · Server example: `vmi3450363` · IP example: `169.58.40.94` · Updated August 2026

Use this checklist whenever you deploy a **new app** onto the shared Contabo Ubuntu host. Do not invent a second proxy stack — join the existing **Caddy + Postgres + Docker network**.

1. Architecture (how this server works)

Component	Location / name	Role
Reverse proxy + HTTPS	<code>/opt/caddy/</code> · container <code>caddy</code>	Terminates TLS, routes domains to app containers
Postgres	container <code>postgres</code> (image <code>postgres:16</code>)	Shared DB server; each app gets its own database + user
Docker network	<code>caddy</code> (external)	Apps talk to <code>caddy</code> and <code>postgres</code> by container name
App folder	<code>/opt/<appname>/</code>	Server-only <code>.env</code> + <code>docker-compose.yml</code>
App container	e.g. <code>easyfuel</code> , <code>inspect360</code>	No public ports — only reachable via Caddy on the Docker network

Rule: App containers must *not* publish host ports like `5002:5002` . Caddy reaches them as `http://<container>:<port>` inside network `caddy` .

2. Before you start — decide these names

Item	Example	Your value
App short name (folder / container)	<code>myapp</code>	_____
Public domain	<code>portal.myapp.ai</code>	_____
Internal app port	<code>5000</code> or <code>5002</code>	_____
Postgres database name	<code>myapp</code>	_____
Postgres app user	<code>myapp_app</code>	_____
GHCR image	<code>ghcr.io/USER/myapp:latest</code>	_____
Server path	<code>/opt/myapp</code>	_____

3. One-time checklist (new project)

- ☐ Create DNS **A record**: domain → Contabo public IP (wait until it resolves).
- ☐ Create Postgres database + user + grants (see §4).
- ☐ Create `/opt/<app>/` with `.env` and `docker-compose.yml` (see §5).
- ☐ Add Caddy site block and reload Caddy (see §6).
- ☐ Add GitHub Actions secrets + deploy workflow (see §7).
- ☐ Build/push first image (or push to `main`) and confirm container is healthy.
- ☐ Restore DB dump / run migrations if needed (see §8).
- ☐ Copy file storage into the Docker volume if the app stores uploads (see §9).
- ☐ Update payment/webhook/callback URLs to the new HTTPS domain.
- ☐ Smoke-test: HTTPS loads, login works, API health, webhooks.

4. Create database on shared Postgres

Admin user on this host is often `creativecloud` (not always `postgres`). Confirm first:

```
docker exec -it postgres psql -U creativecloud -d postgres -c '\du'
```

Then create DB + role (replace names/password):

```
docker exec -it postgres psql -U creativecloud -d postgres <<'SQL'
CREATE USER myapp_app WITH PASSWORD 'STRONG_PASSWORD_HERE';
CREATE DATABASE myapp OWNER myapp_app;
GRANT ALL PRIVILEGES ON DATABASE myapp TO myapp_app;
\c myapp
GRANT ALL ON SCHEMA public TO myapp_app;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT ALL ON TABLES TO myapp_app;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT ALL ON SEQUENCES TO myapp_app;
SQL
```

App `DATABASE_URL` host must be `postgres` (Docker DNS), not `localhost` and not the public IP.

```
DATABASE_URL=postgresql://myapp_app:STRONG_PASSWORD_HERE@postgres:5432/myapp
```

5. App directory on the server

```
mkdir -p /opt/myapp
nano /opt/myapp/.env
nano /opt/myapp/docker-compose.yml
```

5.1 Typical `.env` keys

```
NODE_ENV=production
PORT=5000
HOST=0.0.0.0
TRUST_PROXY=1
DATABASE_URL=postgresql://myapp_app:...@postgres:5432/myapp
PUBLIC_APP_URL=https://portal.myapp.ai
SESSION_SECRET=...
# + any API keys, Resend, payment gateways, etc.
```

Never commit the server `.env`. Keep a private password vault copy.

5.2 Minimal `docker-compose.yml` pattern

```
services:
  myapp:
    image: ${MYAPP_IMAGE:-ghcr.io/USER/myapp:latest}
    container_name: myapp
    restart: unless-stopped
    env_file: .env
    environment:
      NODE_ENV: production
      PORT: "5000"
      HOST: "0.0.0.0"
      TRUST_PROXY: "1"
    volumes:
      - myapp_storage:/app/storage # only if needed
    # NO ports: section
    networks:
      - caddy
    healthcheck:
      test: ["CMD", "node", "-e", "fetch('http://127.0.0.1:5000/').then(r=>process.exit(r.ok?0:1))"]
      interval: 30s
      timeout: 5s
      retries: 3
      start_period: 40s

volumes:
  myapp_storage:

networks:
  caddy:
    external: true
```

6. Caddy (HTTPS + reverse proxy)

Edit shared config carefully:

```
nano /opt/caddy/Caddyfile
```

Add a site block (container name + internal port must match compose):

```
portal.myapp.ai {  
    reverse_proxy myapp:5000  
}
```

Reload (do not recreate blindly if other sites are live):

```
docker exec caddy caddy reload --config /etc/caddy/Caddyfile  
# or from /opt/caddy:  
# docker compose exec caddy caddy reload --config /etc/caddy/Caddyfile
```

Confirm:

```
curl -I https://portal.myapp.ai  
docker ps --filter name=myapp  
docker logs --tail 80 myapp
```

7. GitHub Actions / GHCR deploy

7.1 Repo secrets

Secret	Purpose
CONTABO_HOST	Server IP / hostname
CONTABO_USER	Usually <code>root</code>
CONTABO_SSH_KEY	Private deploy key
GHCR_PULL_TOKEN	Optional PAT with <code>read:packages</code> if image is private
Build-time tokens	e.g. <code>VITE_*</code> baked into SPA

7.2 Workflow outline

1. On push to `main`: build Docker image → push to `ghcr.io/...`
2. SSH to Contabo → `docker login ghcr.io` → `docker pull`
3. `docker compose up -d --force-recreate` inside `/opt/<app>`

Day-to-day code deploys: `git push origin main`. Env-only changes: edit `/opt/<app>/.env` then recreate the container. DB schema/data is *not* automatic unless you add a migration step.

8. Database migration / restore

1. Dump from old host: `pg_dump` (custom `-Fc` or plain SQL).
2. `scp` dump to Contabo.
3. Restore, e.g.:

```
docker cp ./backup.sql postgres:/tmp/backup.sql
docker exec -i postgres psql -U creativecloud -d myapp < /tmp/backup.sql
# or pg_restore for custom format
```

Then verify table ownership/grants for `myapp_app` (fix "permission denied for table ..." errors).

9. File storage (uploads)

1. Zip storage on the old server.
2. Upload to Contabo (WinSCP / `scp` / Filebin).
3. Copy into the named Docker volume:

```
docker volume ls | grep myapp
# Example volume name: myapp_myapp_storage

mkdir -p /tmp/myapp-storage
unzip -o /tmp/storage.zip -d /tmp/myapp-storage
# If zip contains a nested "storage/" folder:
SRC=/tmp/myapp-storage/storage

docker run --rm \
  -v myapp_myapp_storage:/data \
  -v "$SRC":/backup:ro \
  alpine sh -c "cp -a /backup/. /data/ && ls -la /data"

cd /opt/myapp && docker compose up -d --force-recreate
```

10. Post-deploy verification

- ☐ `https://portal....` loads (valid certificate).
- ☐ Login / session cookies work (`TRUST_PROXY=1` , Caddy forwards proto).
- ☐ `docker ps` shows container healthy.
- ☐ App can read/write DB (no permission errors in logs).
- ☐ Uploads load if storage was migrated.
- ☐ External webhooks (payments, email) point at the new HTTPS URLs.
- ☐ Old server DNS removed / app stopped after cutover.

11. Useful commands (cheat sheet)

```
# Status
docker ps
docker logs --tail 100 myapp
docker exec caddy caddy reload --config /etc/caddy/Caddyfile

# Env change
nano /opt/myapp/.env
cd /opt/myapp && docker compose up -d --force-recreate

# DB shell
docker exec -it postgres psql -U myapp_app -d myapp

# Pull latest image manually
cd /opt/myapp
docker pull ghcr.io/USER/myapp:latest
MYAPP_IMAGE=ghcr.io/USER/myapp:latest docker compose up -d --force-recreate
```

12. Existing apps on this host (reference)

App	Path	Domain (example)	Notes
EasyFuel	/opt/easyfuel	portal.easyfuel.ai	Port 5002 · volume easyfuel_easyfuel_storage
Inspect360	/opt/inspect360	portal.inspect360.ai	Port 5000 · image often inspect360:local

13. Common mistakes

- Using `localhost` for Postgres from inside a container → use host `postgres`.
- Publishing app ports on the host → conflicts and bypasses Caddy.
- `PUBLIC_APP_URL` with `http://` → payment redirects / cookies break; use `https://`.
- Editing Caddyfile incorrectly → can take down other sites; always reload, then test all domains.
- Forgetting DB grants → schedulers fail with `permission denied for table ...`.
- Local Windows `.env` using Docker paths like `/app/storage` → wrong for `npm run dev`; use `./storage` locally.